MIDTERM LABORATORY ACTIVITY

AI-Assisted Java Object-Oriented Application Development

“Personalized Productivity Planner Documentation”

Student Name	Abendanio, Angelika Joy A.
Section	TA21
Date	October 9, 2026
Instructor	Dr. Hadji Tejuco

	
Abendanio, Angelika Joy A.	ChatGPT

I. PROJECT TITLE: Personalized Productivity Planner

II. Description

The Personalized Productivity Planner is a single user Java console application for planning one day. The user can enter tasks under four categories: College, Career, Hobby, and Chores and each with a start time and an end time. The program rejects overlapping tasks, shows the day in time order, and shows the free time between tasks, so the user can see how the whole day is being used. It also has a small Motivation Generator and it saves the schedule to a text file so the plan is still there when the program is reopened.

III. Problem Statement

As a college student, I manage my academic responsibilities together with career related activities, hobbies, and household chores. I often have difficulty managing my time because I have a lot of tasks, distractions, and my habit of treating the rest of the day as completely free after I finish my required work. I mainly rely on reminders for single tasks and deadlines, which does not show me how my whole day is used. Because of this, I get tired from studying, doom scroll, and struggle to make time for personal and career activities.

The Personalized Productivity Planner is a single user console application that helps me organize the activities I need to do in the current day. It lets me enter tasks by category with a start time and an end time, rejects overlapping tasks, and shows the free time between my tasks. By seeing my schedule and my available time, I can organize my activities and become more aware of how I use my time.

IV. Target Users

The program is made for one user which is me. I am a college student who has to manage school work, career related activities, hobbies, and household chores in the same day, and I struggle to fit personal and career activities around college.

My main needs are:

- A way to see my whole day in one place, not just reminders for single tasks.
- A warning when two tasks overlap.
- A clear list of the day in time order.
- A way to see my free time, so I do not treat it as empty time.
- A saved plan, so I do not have to type it again after closing the program.

V. Objectives of the Application

General Objective: To help the user organize and visualize their activities for the current day, so they can see how their time is used and reflect on their own productivity.

Specific Objectives:

- Let the user enter daily activities under the categories COLLEGE, CAREER, HOBBY, and CHORES.
- Let the user give each activity a start time and an end time in the 24-hour HH:mm format.
- Detect overlapping tasks and reject them, along with empty titles and invalid times.
- Display the schedule in time order.
- Find and display the free time between tasks inside a planning period chosen by the user, so it can be filled by adding new tasks.
- Let the user remove a task.
- Provide a simple Motivation Generator that shows a random motivational message, so the program is less boring and gives some encouragement.
- Save the schedule to a file and load it again, so the day's plan stays when the program is closed and reopened.
- Help the user become more aware of how the day is used. Reflection is the benefit of seeing the schedule, not a separate feature.

VI. Scope & Limitations

The Personalized Productivity Planner is a single user Java console application for organizing activities within one day.

Scope:

- A single user planning the current day. Tasks have start and end times but no dates.
- Four categories: College, Career, Hobby, Chores.
- Times in the 24 hour HH:mm format. Empty titles, wrong formats, end times that are not after the start time, and overlapping tasks are rejected. Back to back tasks, such as 09:00 to 10:00 and 10:00 to 11:00, are allowed.
- Removing a task by title, ignoring uppercase and lowercase.
- Viewing the schedule sorted by start time.
- Showing free time inside a planning period chosen by the user.
- A random motivational message.

- Saving the schedule to schedule.txt and loading it when the program starts, so the current day's plan is kept when the program is reopened.

Limitations:

- The program does not include user accounts, graphical interface, notifications, automatic task recommendations, task priorities, completion tracking, weekly or long term schedules, or online synchronization.
- The program cannot guarantee that the user becomes productive or stays motivated. Productivity depends on the person and cannot be measured or controlled by the program.
- The planning period for free time (ex: 05:00 to 21:00) is entered when using Show Free Time. It is not set when the program starts.
- The program shows free time but it does not ask the user to fill it. The user adds tasks from the main menu.
- There is no reset option. Tasks are removed one at a time.
- A task cannot cross midnight. A task from 23:00 to 00:00 is rejected.
- A title with the | character is saved but not loaded back, because | separates the fields in schedule.txt (Test 19).
- The time 24:00 is accepted and treated as 00:00 (Test 20).
- If two tasks have the same title, removing that title deletes only the first one, and the program does not say which one (Test 22).
- A task cannot be edited. It must be removed and added again.
- If someone edits schedule.txt by hand and creates overlapping tasks, the later task is left out when the program loads.
- The screen is not cleared between actions, so the console output gets longer as the program is used.
- The program is console only and uses one save file.

VII. ChatGPT Brainstorming Summary

I wanted to build a program to help with my time management. I used ChatGPT to brainstorm before writing any code and we merged our ideas into one plan. It became a console planner where I enter tasks by category with start and end times. I can see my day in order, find free time and get a motivational message.

ChatGPT asked me questions to narrow down my problem statement, objectives and scope instead of writing them for me. My answers showed the real problem. I have many

responsibilities and I rely on reminders for single tasks. After I finish my work I treat the rest of the day as free, so I end up doom scrolling.

Two decisions came out of this. First, ChatGPT told me not to add file saving because we described a day to day planner. I kept it so my plan stays when I close and reopen the program. Second, ChatGPT said that practicing being productive is my personal goal and the program cannot measure it. We changed the main objective to helping the user organize and see the day. I decided that reflection is a benefit of seeing the schedule and not a feature.

For the design, ChatGPT first suggested an abstract Task class with four subclasses called CollegeTask, CareerTask, HobbyTask and ChoreTask. It also suggested a Schedulable interface, a class to manage the tasks and a main class for the menu. It asked if the categories should be subclasses or just an enum. It warned me not to make subclasses only to meet the inheritance requirement.

Later, ChatGPT split the program into Task, DailyPlanner, ScheduleStorage, MotivationGenerator and PlannerApp. It also suggested a Category enum. I compared both designs. I kept the subclasses because the assignment needs real inheritance, overriding and polymorphism. I used the class split from the later version.

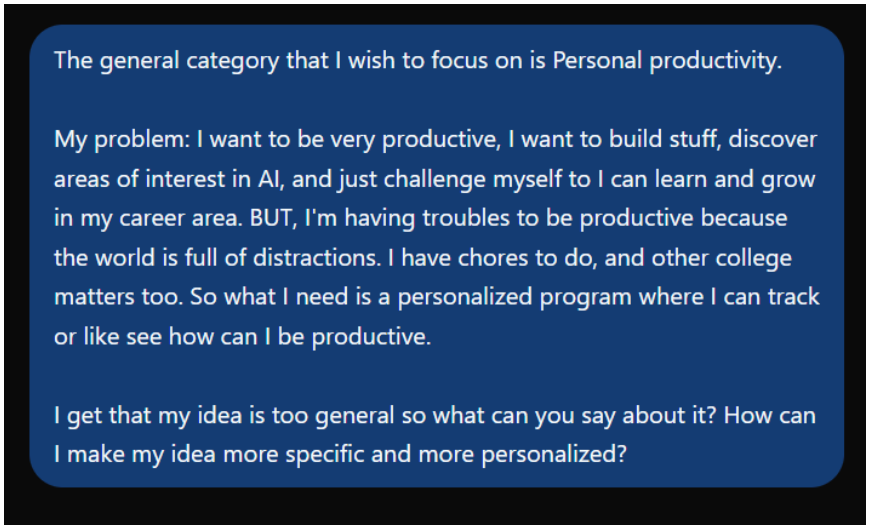


Fig 1. My first question to ChatGPT

VIII. Final Proposed Solution

The final solution is a menu based Java console program. When it starts it loads any saved tasks from schedule.txt. The user then chooses from six options:

1. Add Task: choose a category, then enter a title, a start time, and an end time.
2. Remove Task: delete a task by its title.
3. View Schedule: show all tasks in order of start time.
4. Show Free Time: show the open time between tasks inside a planning period.
5. Motivation: show a random motivational message.
6. Save and Exit: save the schedule and close the program.

IX. Identified Classes, Attributes, Methods, and Responsibilities

Class	Attributes	Methods	Responsibility
Schedulable (interface)	n/a	+ getStartTime(): LocalTime; + getEndTime(): LocalTime; + overlaps(other: Schedulable): boolean	Sets the rule for anything that has a start time and an end time and can be checked for overlap.
Task (abstract class)	- title: String; - startTime: LocalTime; - endTime: LocalTime	# Task(title, startTime, endTime); + getTitle(): String; + setTitle(title: String): void; + getStartTime(): LocalTime; + getEndTime(): LocalTime; + overlaps(other: Schedulable): boolean; + getCategory(): String (abstract); + displayTask(): void	Holds the data and behavior every task shares: validates the title and times, checks overlap, and prints the task.
CollegeTask	n/a	+ CollegeTask(title, startTime, endTime); + getCategory(): String;	College task. Returns COLLEGE and prints with the [COLLEGE] tag.

		+ displayTask(): void	
CareerTask	n/a	+ CareerTask(title, startTime, endTime); + getCategory(): String; + displayTask(): void	Career task. Returns CAREER and prints with the [CAREER] tag.
HobbyTask	n/a	+ HobbyTask(title, startTime, endTime); + getCategory(): String; + displayTask(): void	Hobby task. Returns HOBBY and prints with the [HOBBY] tag.
ChoreTask	n/a	+ ChoreTask(title, startTime, endTime); + getCategory(): String; + displayTask(): void	Chore task. Returns CHORES and prints with the [CHORES] tag.
DailyPlanner	- tasks: ArrayList<Task>	+ DailyPlanner(); + addTask(task: Task): boolean; + removeTask(title: String): boolean; + sortTasks(): void; + getFreeTime(dayStart: LocalTime, dayEnd: LocalTime): void; + getTasks(): ArrayList<Task>	Keeps the list of tasks. Rejects overlapping tasks, removes tasks, sorts them, and prints free time.
ScheduleStorage	- FILE_NAME: String (constant, schedule.txt); - TIME_FORMAT: DateTimeFormatter	+ saveSchedule(tasks: ArrayList<Task>): void; + loadSchedule(): ArrayList<Task>; - createTask(category, title, startTime, endTime): Task	Saves tasks to schedule.txt and loads them back. Rebuilds the correct subclass and skips bad lines.
MotivationGenerator	- messages: String[]; - random: Random	+ generateMessage():	Returns a random motivational

		String	message.
PlannerApp	- planner: DailyPlanner; - storage: ScheduleStorage; - motivation: MotivationGenerator ; - scanner: Scanner; - timeFormatter: DateTimeFormatter	+ PlannerApp(); + main(args: String[]): void; + run(): void; - showMenu(): void; - addTask(): void; - removeTask(): void; - viewSchedule(): void; - showFreeTime(): void; - readTime(prompt: String): LocalTime; - createTask(category, title, startTime, endTime): Task	Runs the console menu, reads the user's input, and connects the other classes.

X. UML Class Diagram

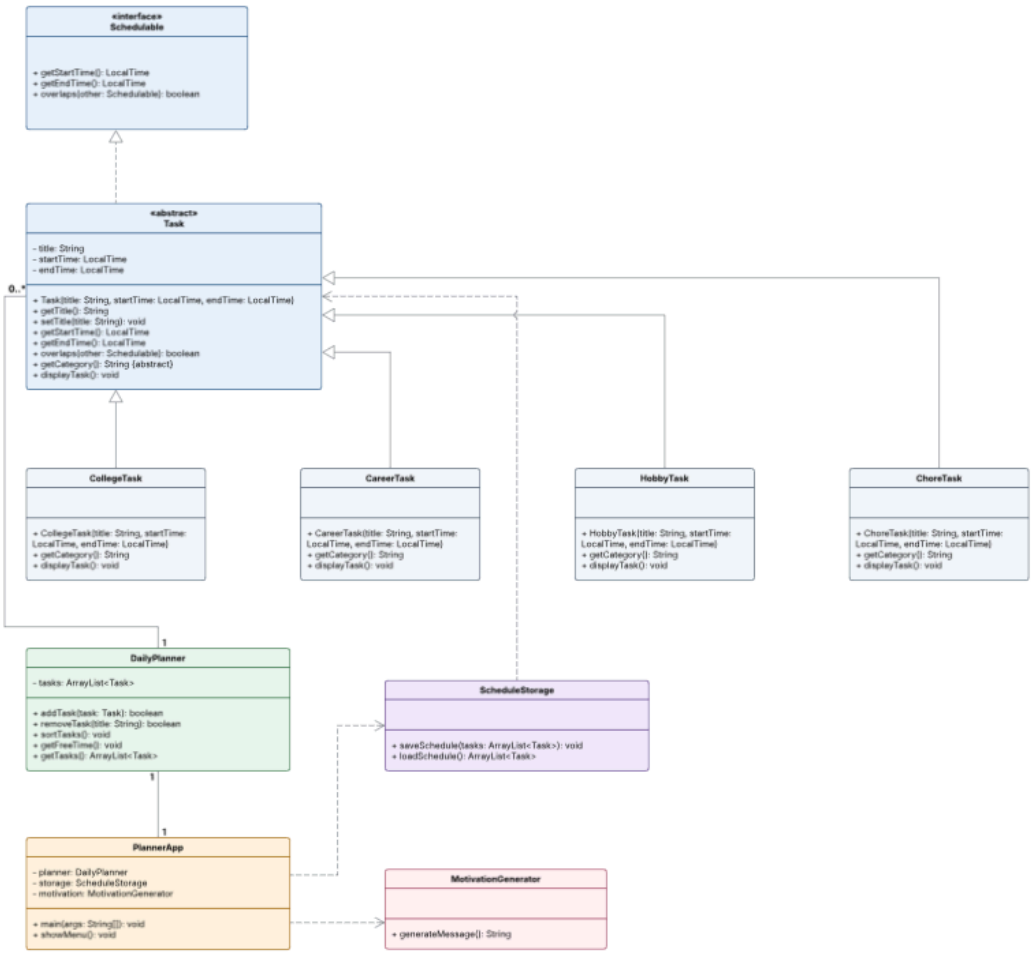


Fig 2. UML class diagram of the Personalized Productivity Planner.

XI. Explanation of Class Relationships

Class A	Relationship Type (is-a/has-a)	Class B	Why
Task	is-a (implements)	Schedulable	A Task has a start time and an end time, so it can be checked for overlap. It promises the three Schedulable methods.
CollegeTask	is-a (extends)	Task	It's a kind of task

CareerTask	is-a (extends)	Task	It’s a kind of task.
HobbyTask	is-a (extends)	Task	It’s a kind of task.
ChoreTask	is-a (extends)	Task	It’s a kind of task.
DailyPlanner	has-a (1 to 0..*)	Task	The planner holds zero or many tasks in its list. A planner is not a task.
PlannerApp	has-a (1 to 1)	DailyPlanner	It owns one planner and sends the user's actions to it.
PlannerApp	has-a (uses)	ScheduleStorage	App calls it to save and load the schedule.
PlannerApp	has-a (uses)	MotivationGenerator	App calls it to get a motivational message.
ScheduleStorage	uses	Task	It reads the data of Task objects to save them and creates new ones when loading.

XII. Explanation of Encapsulation

Encapsulation keeps data private so other classes can only reach it through methods. In my program, Task keeps title, startTime, and endTime as private. DailyPlanner also keeps its tasks list private.

Other classes use getters like getTitle() and getStartTime() to read the data. The only setter is setTitle(), and it makes sure the title is not empty. The constructor also checks the data, and it uses this.startTime = startTime; to save the value in the object. It is protected, so only the subclasses can use it. Because of this, a task with an empty title or a wrong time can never be created.

XIII. Explanation of Inheritance

Inheritance lets a subclass reuse the code of a parent class. Task is the parent because every task has a title, a start time, and an end time. I wrote this once in Task so I don't repeat it four times.

CollegeTask, CareerTask, HobbyTask, and ChoreTask all extends Task. Each one is a kind of task, so this is an is-a relationship. They get getTitle(), getStartTime(), getEndTime(), and overlaps() from Task. Each constructor uses super(title, startTime, endTime) to send the values to the Task constructor, which checks them.

XIV. Explanation of Polymorphism

Polymorphism allow the same method call where it can give different results depending on the object. DailyPlanner keeps all tasks in one ArrayList<Task> but the objects inside are CollegeTask, CareerTask, HobbyTask, and ChoreTask.

Each subclass uses @Override for getCategory() and displayTask(). So displayTask() prints [COLLEGE] for a college task and [HOBBY] for a hobby task. In PlannerApp.viewSchedule(), the loop calls task.displayTask() on every task, and Java runs the version that belongs to each object (Test 5). My overridden methods do not call super.methodName(). The only super I use is super(...) in the constructors.

XV. Explanation of Absatraction

Abstraction means hiding the details and showing only the general idea. Task is abstract because a task without a category makes no sense in my planner. I can create a CollegeTask, but I cannot write new Task(...).

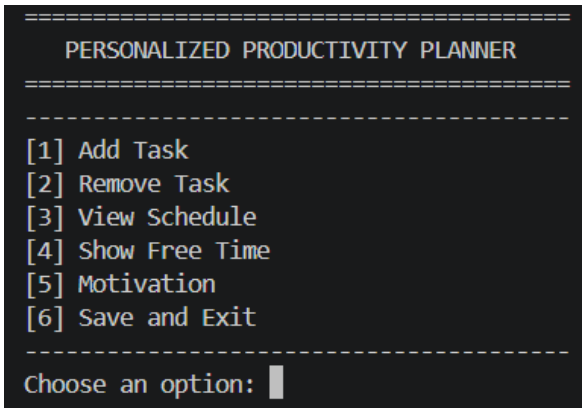
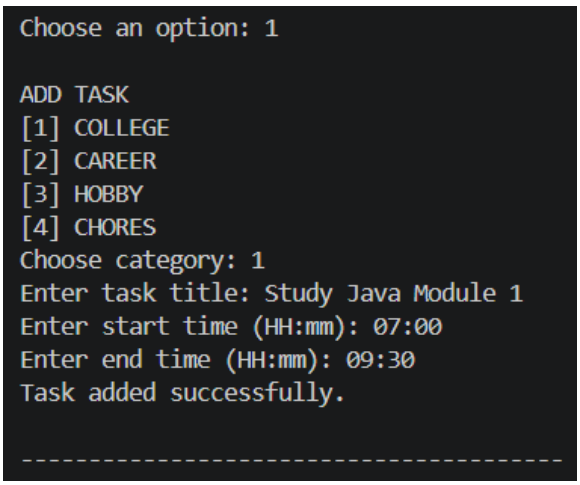
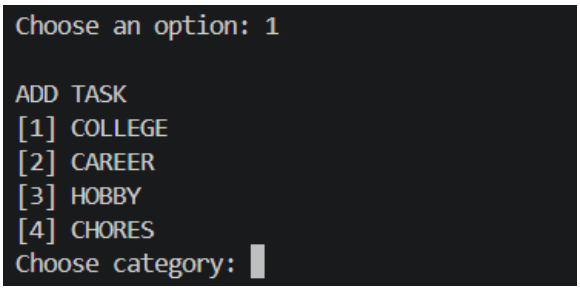
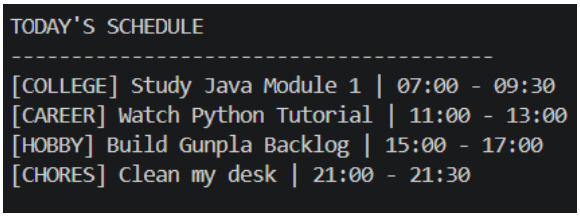
Task has one abstract method, getCategory(), so every subclass must write its own. The other methods in Task already have code, and the subclasses reuse them. Because of this, DailyPlanner and PlannerApp only need to know about Task.

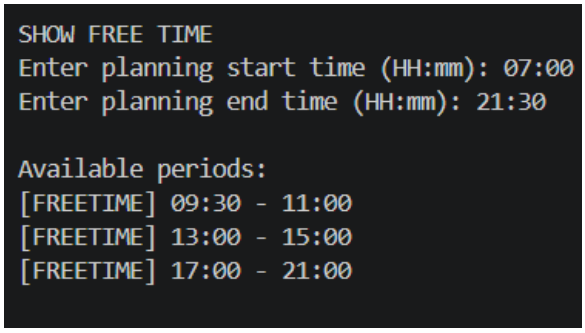
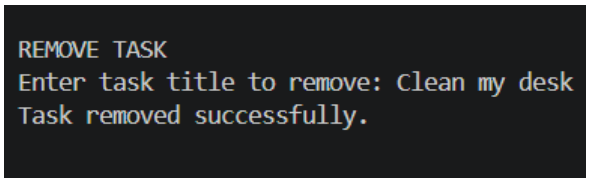
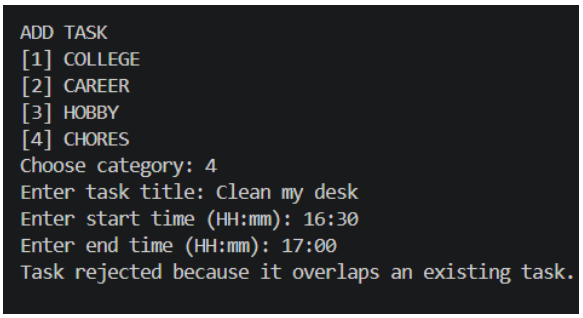
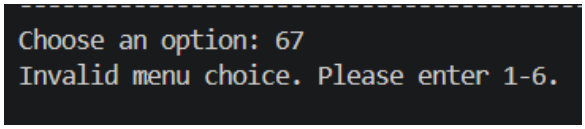
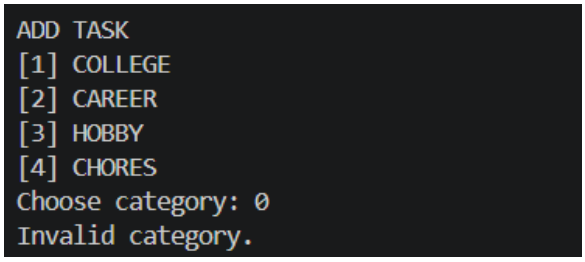
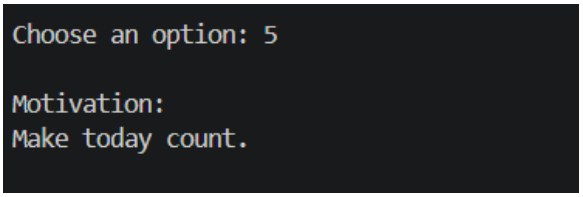
XVI. Explanation of Interface

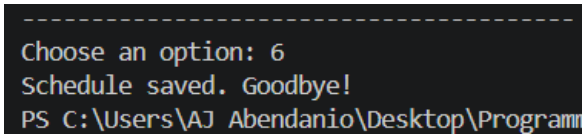
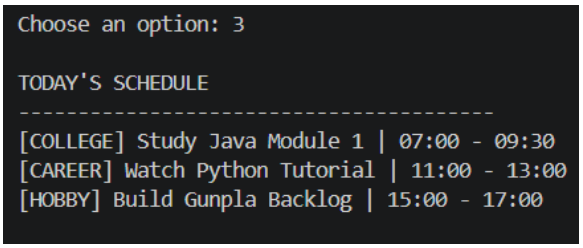
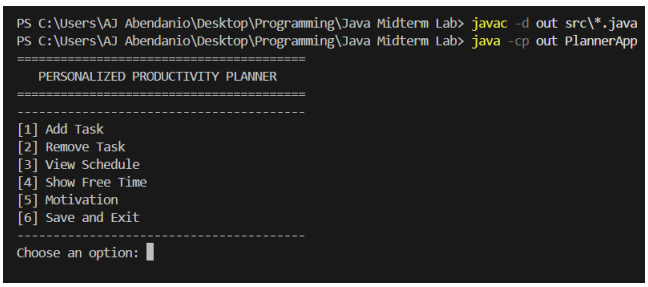
Schedulable is an interface and it lists three methods: getStartTime(), getEndTime(), and overlaps(Schedulable other). It only sets the rules and has no code inside.

Task uses implements Schedulable and writes the code for all three methods with @Override. The four subclasses get them from Task. An abstract class like Task can hold fields and shared code, while an interface only describes what a class must do. A class can extend only one parent, but it can implement many interfaces. In my program, extends is for the task hierarchy, implements is for the interface, and @Override marks the methods that replace a parent or interface method.

XVI. Screenshots of Program Execution

Figure #	Description	Screenshot
1	Main menu	
2	Adding a task	
3	Four task categories	
4	Viewing the Schedule	

5	Showing Free Time	
6	Removing a task	
7	Overlap validation	
8	Invalid input handling	 
9	Motivation feature	

10	Saving schedule	
11	Loading saved schedule	
12	Compilation and execution	

XVII. Test Case Table

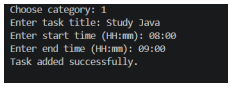
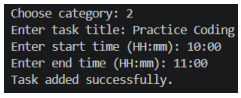
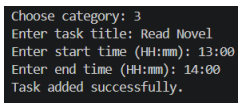
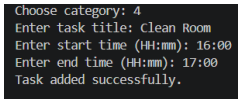
Name: Angelika Joy Abendanio
Application Type: Console-Based Application
Development Environment: Visual Studio Code

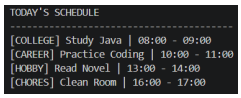
I. TEST INFORMATION

Field	**Details**
Project Name	*Personalized Productivity Planner*
Testing Environment	*VS Code Terminal*
Java Version	*Java 17*
Test Date	*October 9, 2026*
Tested By	*Angelika Abendanio*
Total Test Cases	*22*

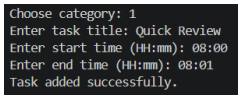
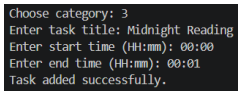
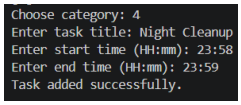
Total Passed	*19*
Total Failed	*3*

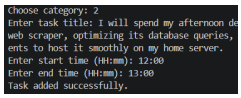
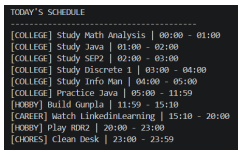
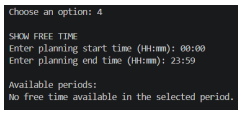
II. VALID INPUT AND SUBCLASS TESTING

Test #	Feature	Input	Expected Result	Actual Result	Pass/ Fail	Screenshot
1	Add College Task	1. Select [1] Add Task. 2. Choose College category. 3. Title: Study Java. 4. Start: 08:00. 5. End: 09:00.	College task is successfully added with the correct title, category, and time.	*Same as expected*	***Pass***	
2	Add Career Task	1. Select [1]. 2. Choose Career category. 3. Title: Practice Coding. 4. Start: 10:00. 5. End: 11:00.	Career task is successfully added to the schedule.	*Same as expected*	***Pass***	
3	Add Hobby Task	1. Select [1]. 2. Choose Hobby category. 3. Title: Read Novel. 4. Start: 13:00. 5. End: 14:00.	Hobby task is successfully added to the schedule.	*Same as expected*	***Pass***	
4	Add Chore Task	1. Select [1]. 2. Choose Chore category. 3. Title:	Chore task is successfully added to the schedule.	*Same as expected*	***Pass***	

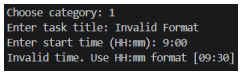
		Clean Room. 4. Start: 16:00. 5. End: 17:00.				
5	Mixed Subclasses / Polymorphism	1. Add one task from each of the four categories at different times. 2. Select [3] View Schedule.	All four task types appear in the schedule with their correct category-specific information.	*Same as expected*	***Pass***	

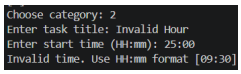
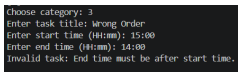
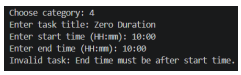
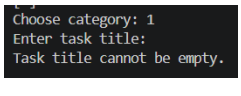
III. MINIMUM AND MAXIMUM VALUES

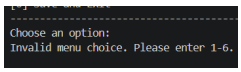
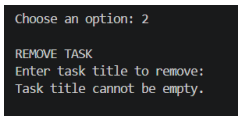
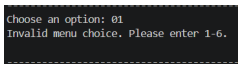
Test #	Feature	Input	Expected Result	Actual Result	Pass/ Fail	Screenshot
6	Minimum Task Duration	1. Select [1]. 2. Choose College. 3. Title: Quick Review. 4. Start: 08:00. 5. End: 08:01.	A task lasting exactly one minute is accepted.	*Same as expected*	***Pass***	
7	Earliest Start Time	1. Select [1]. 2. Choose Hobby. 3. Title: Midnight Reading. 4. Start: 00:00. 5. End: 00:01.	Task starting at 00:00 is accepted.	*Same as expected*	***Pass***	
8	Latest End Time	1. Select [1]. 2. Choose Chore. 3. Title: Night	Task ending at 23:59 is accepted.	*Same as expected*	***Pass***	

		Cleanup. 4. Start: 23:58. 5. End: 23:59.				
9	Very Long Task Title	1. Select [1]. 2. Choose Career. 3. Enter a title of at least 200 characters. 4. Start: 12:00. 5. End: 13:00.	The program accepts the nonempty title and displays or stores it without unintended truncation.	*Same as expected*	***Pass***	
10	Full Day of Back-to-Back Tasks	1. Begin with an empty schedule. 2. Add consecutive tasks covering 00:00–01:00, 01:00–02:00, and so on through 22:00–23:00. 3. Add a final task 23:00–23:59. 4. View Schedule and Show Free Time.	All 24 tasks are accepted without overlap errors. No free-time interval appears between consecutive tasks.	*Same as expected*	***Pass***	 

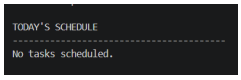
IV. INVALID AND EMPTY INPUT

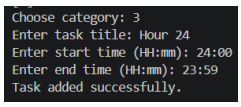
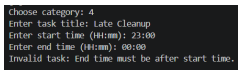
Test #	Feature	Input	Expected Result	Actual Result	Pass/ Fail	Screenshot
11	Invalid Time Format	1. Select [1]. 2. Choose College. 3. Title: Invalid	Time format is rejected because it does not follow the	*Same as expected*	***Pass***	

		Format. 4. Enter start time 9:00. 5. If prompted, enter end time 10:00.	required HH:mm format.			
12	Invalid Hour	1. Select [1]. 2. Choose Career. 3. Title: Invalid Hour. 4. Start: 25:00. 5. End: 26:00, if prompted.	Invalid hours are rejected and the task is not added.	*Same as expected*	***Pass***	
13	End Before Start	1. Select [1]. 2. Choose Hobby. 3. Title: Wrong Order. 4. Start: 15:00. 5. End: 14:00.	Task is rejected because the end time is earlier than the start tim	*Same as expected*	***Pass***	
14	Equal Start and End	1. Select [1]. 2. Choose Chore. 3. Title: Zero Duration. 4. Start: 10:00. 5. End: 10:00.	Task is rejected because the end time must be later than the start time.	*Same as expected*	***Pass***	
15	Empty Task Title	1. Select [1]. 2. Choose College. 3. Press Enter without typing a title. 4. Enter valid times if requested.	Empty title is rejected and no task is added.	*Same as expected*	***Pass***	

16	Empty Menu Choice	1. Open the main menu. 2. Press Enter without entering a choice.	Empty menu input does not execute any menu action. Program remains usable.	*Same as expected*	***Pass***	
17	Empty Removal Title	1. Add Study Java. 2. Select [2] Remove Task. 3. Press Enter without typing a title.	No task is removed. The program handles the empty removal input without crashing.	*Same as expected*	***Pass***	
18	Incorrect Menu Selections	1. At the main menu, enter 0. 2. Repeat with 7. 3. Repeat with abc. 4. Repeat with @.	Each invalid selection is handled without executing an unintended action or terminating unexpectedly	*Same as expected*	***Pass***	

V. ADDITIONAL EDGE CASES

Test #	**Feature**	**Input**	**Expected Result**	**Actual Result**	**Pass/ Fail**	**Screenshot**
19	Special Character in Title	1. Add College task titled Read | Notes, 08:00 to 09:00. 2. Select [6] Save and Exit. 3. Restart the program. 4. Select [3] View Schedule.	Task is saved and appears again after restarting, with the same title.	*The data got stored in the txt save file but when I ran it again, it showed no tasks scheduled.*	***Fail***	

20	Hour 24 Input	1. Select [1], choose Hobby. 2. Title: Hour 24. 3. Start: 24:00. 4. End: 23:59, if prompted.	24:00 is rejected as an invalid time.	*24:00 got accepted as a valid input and was not flagged as invalid time :<*	***Fail***	
21	Task Crossing Midnight	1. Select [1], choose Chore. 2. Title: Late Cleanup. 3. Start: 23:00. 4. End: 00:00.	Task is either accepted, or rejected with a clear message.	*Task is rejected with a clear message*	***Pass***	
22	Duplicate Titles	1. Add Study Java, 08:00 to 09:00. 2. Add Study Java, 10:00 to 11:00. 3. Select [2] and remove Study Java. 4. View Schedule.	Removal behaves predictably, and the result is clear to the user.	*The first Study Java entry got removed and there is no clear result/message*	***Fail***	

VI. TEST SUMMARY

Summary	**Result**
Total Test Cases Executed	*22*
Passed	*19*
Failed	*3*
Not Tested	-
Overall Testing Result	*86.36% / B+ for Error Handling & Input validation*

Date Completed	*October 9,2026*

XVIII. Problems Encountered During Development

1. I wanted the .class files to be put in the out folder but the compiled files were saved inside src so the .class files appeared next to my .java files. I fixed it by compiling from the project root with javac -d out src*.java and running with java -cp out PlannerApp. I also deleted the stray .class files and added out/ and *.class to .gitignore.

2. The console gets too long and messy and I wanted the screen to clear after each action. I tried a do while loop at first but i couldnt get the correct syntax so it does not clear anything. ChatGPT suggested ANSI escape codes and a Press Enter to continue pause but I decided to keep the original menu loop, so the program does not clear the screen between actions.

3. Failed tests in Testing phase:

    - Test 19, | in a title: the task was saved to schedule.txt but didn't appear after restarting. The file uses | between fields so I believe a title with | makes the line have five parts instead of four and loadSchedule() skips it as a bad line.

    - Test 20, the time 24:00: it was accepted instead of rejected. The HH:mm formatter reads 24:00 as 00:00, which is why a task from 24:00 to 23:59 was accepted.

    - Test 22, duplicate titles: removeTask() deletes only the first match and stops. The program says a task was removed, but not which one.

XIX. How ChatGPT Helped

ChatGPT helped me in these:

- Plan the class structure: the abstract Task, the subclasses, the Schedulable interface, and the jobs of DailyPlanner and PlannerApp.

- It raised the question of subclasses versus an enum which made me think about whether inheritance had a real purpose in my program.

- It asked me questions to narrow down my problem statement, objectives, and scope instead of writing them for me.

- It gave me the important syntaxes in my Java files, which I used as my basis in building the program.

- It explained how to clear the console with ANSI escape codes and why a do-while loop alone does not clear the screen. I did not use this in the final program.
- It gave me a testing template and a documentation template to organize my test cases and this report.

XX. AI Suggestions I Accepted

AI Suggestion	My Evaluation	My Decision
Make Task an abstract class with four subclasses: CollegeTask, CareerTask, HobbyTask, ChoreTask.	Each category is a kind of task, so it is a real is-a relationship. Each subclass can override getCategory() and displayTask() so inheritance has a real purpose.	Accepted.
Create a Schedulable interface with getStartTime(), getEndTime(), and overlaps().	Overlap checking depends only on time, not on the category so it fits an interface that Task implements.	Accepted.
Split the program into DailyPlanner, ScheduleStorage, MotivationGenerator, and PlannerApp.	Each class gets one job, which makes the program easier to read and explain.	Accepted.
Each class gets one job, which makes the program easier to read and explain. Accepted. Treat reflection as the benefit of seeing the schedule, and word the main objective as helping the user organize and visualize the day.	I know program cant measure productivity, so the objective should only claim what the program can actually do.	Accepted.

XXI. AI Suggestions I Rejected

AI Suggestion	My Evaluation	My Decision
Do not add file storage, because the planner is a day by day session planner.	I want my plan to stay when I close and reopen the program so file saving is a need for me.	Rejected. I do file saving with schedule.txt
Use a Category enum instead of subclasses for COLLEGE, CAREER, HOBBY, and CHORES.	I believe enum is simpler but it has no inheritance, overriding, polymorphism, or abstraction, and this project/exam requires them. The subclasses also have different behavior in displayTask().	Rejected. I used subclasses.
Add a SchedulePersistence interface for ScheduleStorage.	ScheduleStorage has only one version so a second interface would be extra. Schedulable already gives my program a good interface.	Rejected. I didn't add it
Read the menu choice with nextInt() and then clear the leftover newline.	nextInt() leaves a newline in the input and fails when the user types letters or symbols. My program reads a whole line, so abc and @ are handled (Test 18)	Modified. I kept nextLine() with a String switch.
Clear the screen with ANSI escape codes and add a Press Enter to continue pause, using a do-while loop.	It could make the console cleaner but when I tried it it wasn't working plus it adds extra code to every menu action and my while (running) loop already works.	Rejected. I removed it and kept my original menu loop.

XXII. What I Personally Learned

This project taught me that OOP is mostly about deciding what belongs where. At first I thought of the categories as like simple labels but when I had to explain inheritance, I saw that each category could be a real subclass of Task that overrides getCategory() and displayTask().

I also learned that polymorphism is easy to see in one place like in my the ArrayList<Task> that holds four different kinds of tasks and calls displayTask() on each one. And lastly, testing taught me that a program that works with a normal input can still fail on edge cases.

My three failed tests showed problems I did not see while coding despite being so sure that it was already fool proof. I also learned practical things, like using javac -d out to keep compiled files out of src and that a do-while loop does not clear the screen by itself.

XXIII. Conclusion

The Personalized Productivity Planner meets its main objective. It lets me add tasks in four categories, blocks overlapping tasks, shows the schedule in time order, finds free time, and saves the schedule to a file. The program also uses the OOP concepts the project required which are encapsulation, inheritance, overriding, polymorphism, abstraction, and an interface.

I ran 22 test cases. 19 passed and 3 failed which is 86.36% passed. The failures were edge cases: a | in a title, the time 24:00, and duplicate titles. They do not stop the normal use of the program and I just listed them as limitations.

The project shows that a simple console program can solve a real planning problem when its classes have clear jobs. The possible improvements are adding dates, allowing task editing, rejecting | in titles, checking for 24:00, and of course finding a way for the console to be cleaner while the program is running.